

Functional Requirement Specification

Stakes holders:

Primary actors:

- Customer: This actor can participate in pre-order coffee, self-order coffee, wait in line, pay for the drink they need and wait to call for the number to pick up their drink.
- Sponsor: This actor can participate in funding the money for open coffee operation, their interest upon the project is how to make sure it is running efficiently
- Designer: This actor can participate in helping design the design to help improve the user interface and user experience to make the app looks good.

Secondary actors:

- Employees: This actor can interact with the customer guide and assign the user through the process of ordering coffee from start to finish, help user register for membership
- Managers: This actor can view reports, can check inventory, and have higher roles than employees
- System: giving details about drinks, inventory availability, user membership points, and drinks information, total income daily

Manager (12)

- Tracking: Manager can view activities (3)
- Adjust sales records (3)
- Track Ingredients (2)
- Manager user info (2)
- Managers must check if employees are correctly converting vouchers or not (2)

Employee (11)

- Log in / Log out: From employee account. Employees can login with their own username and password (2)
- View account: Let employees view their account (2)
- Scan bar code: Let employees scan bar code (represent and simulate payment through credit card) (3)

- Pay by cash: Let the user give back changes and total (2)
- Print receipt: print out the receipt on amount that customers bought from the store (1)
- Help users set up membership accounts (1)

Customer (11)

- Preordering: To order before they come or user can wait and order when other people are in line (2)
- Wait in line: Directly interact with employees to order drinks (3)
- Self ordering: Directly interact with the kiosk system, order the drinks they need (3)
- Pay for the drink: Customers are waiting in line and Self ordering must pay for the drink (2)
- Wait for the call number: Customer sits and waits until their number is being called on the ticket receipt and shows. (1)

System (13)

- Membership point: calculates the point and allow user to use this point to buy drinks and use it as discount for next drinks (3)
- Self – Service ordering: User will be able to order for take away and order without employee (2)
- Payment feature: let user purchase drinks and food, print out a receipt (1)
- Back features selection, let user to go back to where they want to go (1)
- Scan bar code to pay (2)
- Display the GUI, Display menu selection, (3)
- Let user interact with the system, show drinks, food with its prices (1)

The diagram illustrates the functional requirements of a restaurant management system, organized by actor and use case type.

Actors:

- Manager** (Blue smiley face)
- Employee** (Red smiley face)
- Customer** (Yellow smiley face)

Use Cases (Color-coded by actor):

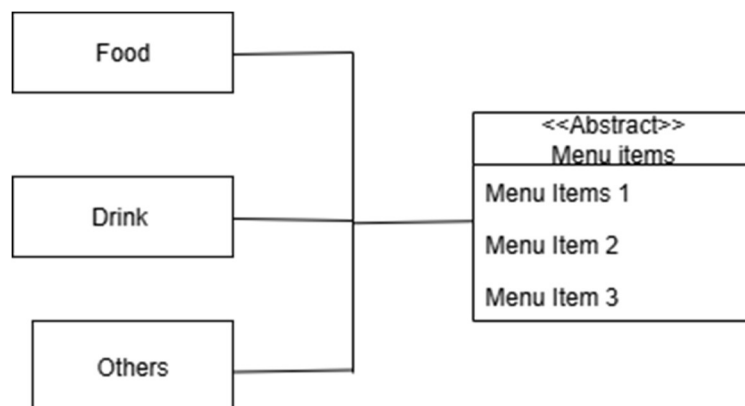
- Manager (Blue):** View Activities, Adjust Sales Records, Track Ingredients, Manage User Info.
- Employee (Red):** Print Receipt, Scan BarCode, Login / Logout, Pay Cash, View Account.
- Customer (Yellow):** Preordering, Self Ordering, Wait in line, Cash, Wait for call, Pay for Drink.
- System (Purple):** Membership Point, auto service, Scan Bar Code, Check Voucher validity, Register Membership, Show Drinks & Foods, Provides payment features, Display Interaction.

Relationships (Connections):

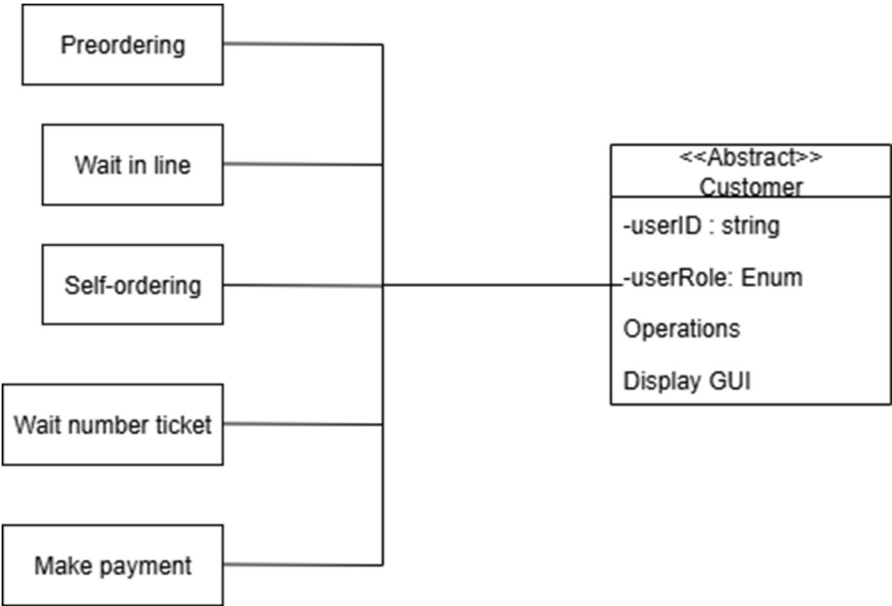
- Manager to System:** View Activities, Adjust Sales Records, Track Ingredients, Manage User Info.
- Employee to System:** Print Receipt, Scan BarCode, Login / Logout, Pay Cash, View Account.
- Customer to System:** Preordering, Self Ordering, Wait in line, Cash, Wait for call, Pay for Drink.
- Internal System Flow:** Add Food / Drinks & Prices → Show Drinks & Foods; auto service → Scan Bar Code; Check Voucher validity → Register Membership; Provides payment features → Display Interaction.

Scan Bar Code <<include>> Pay for Drink

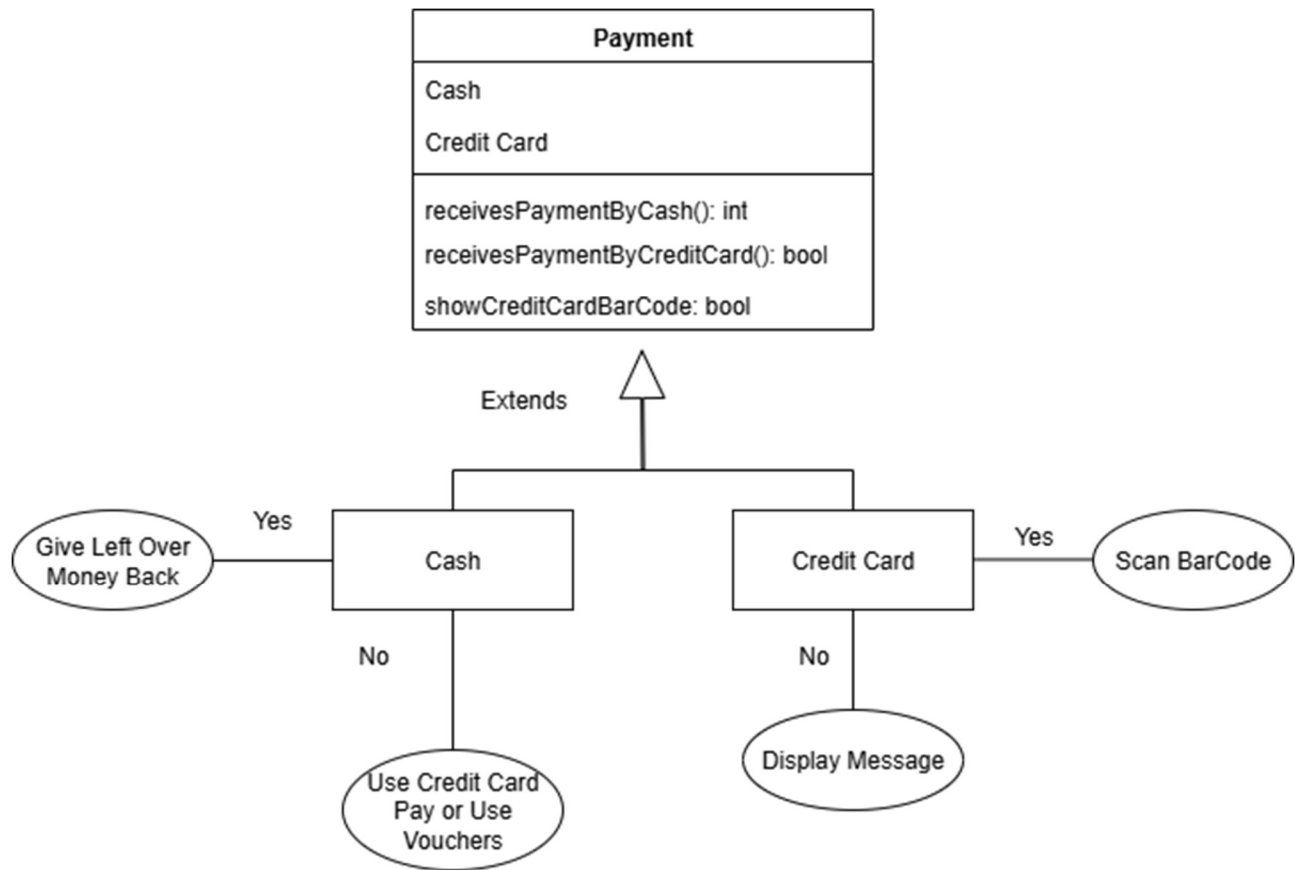
Menu Items



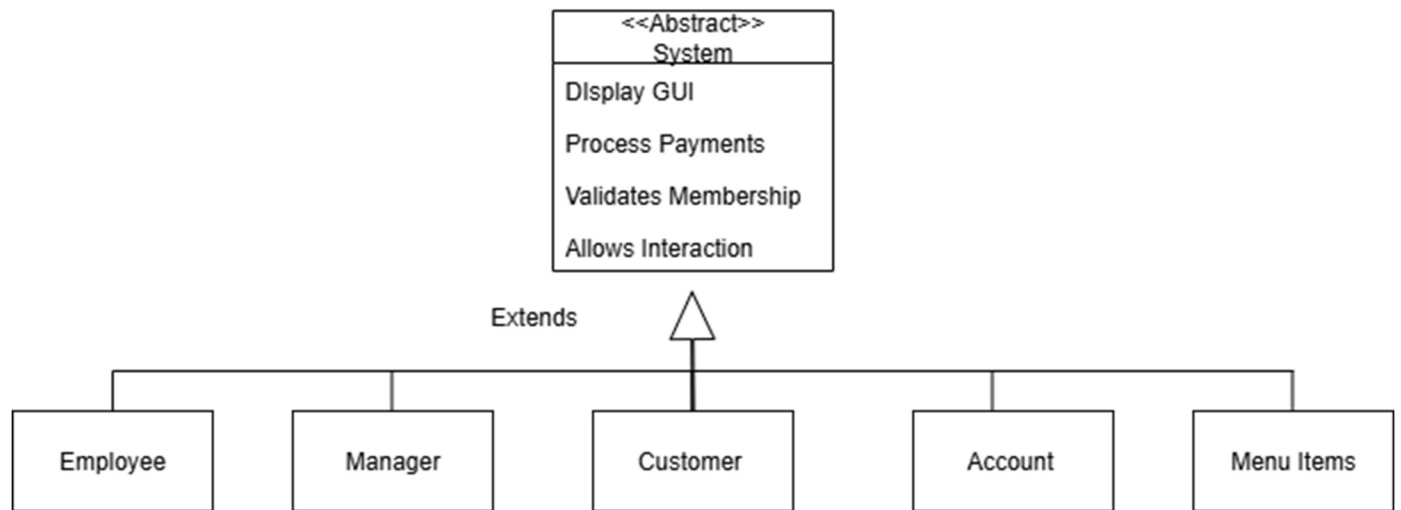
Customer Class



Payment Class



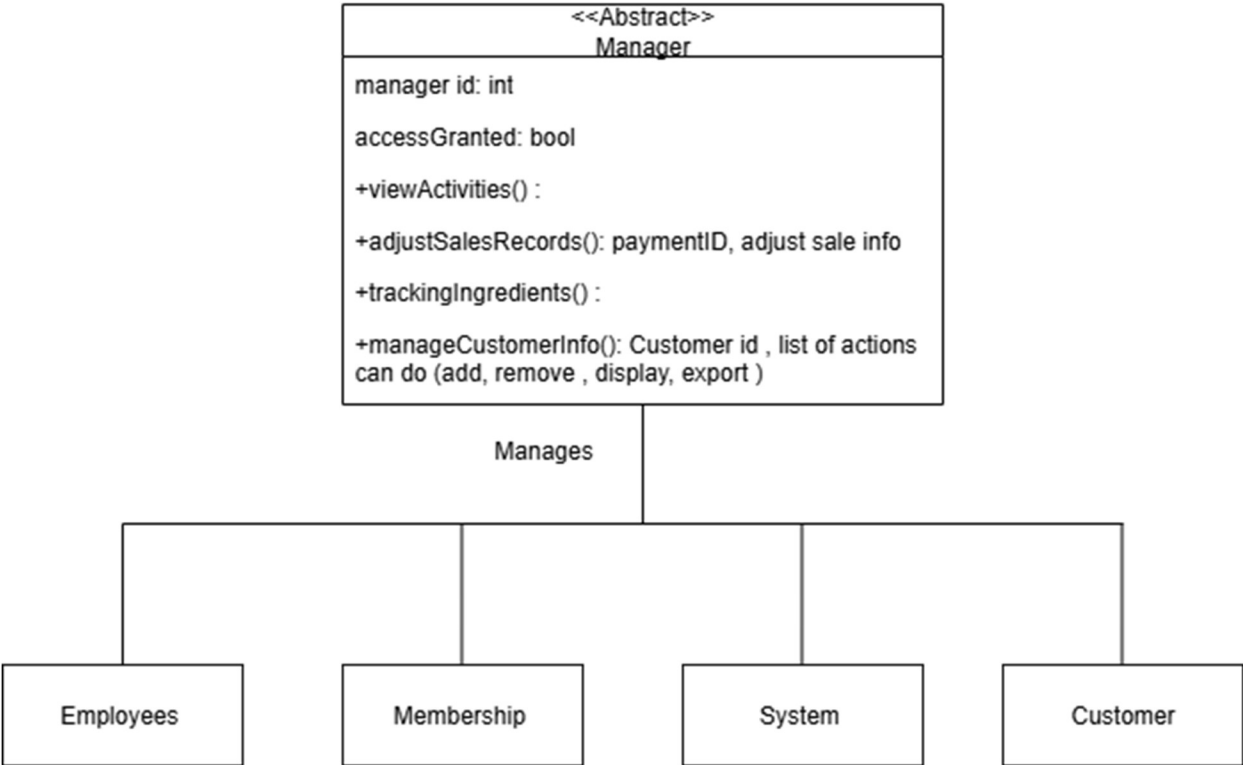
System Class



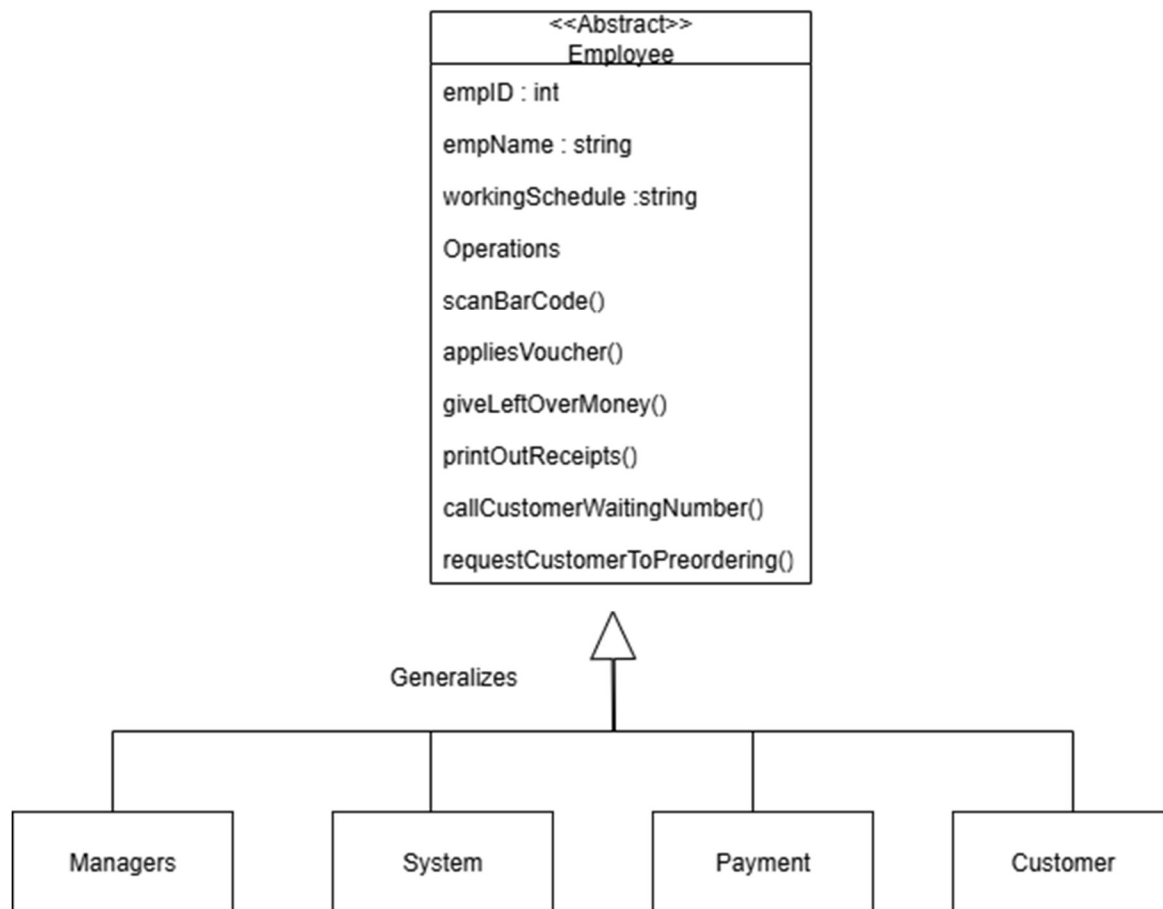
Products Class

Products
food items: id, string, int
drink items: id, string, int getProductItems(): bool

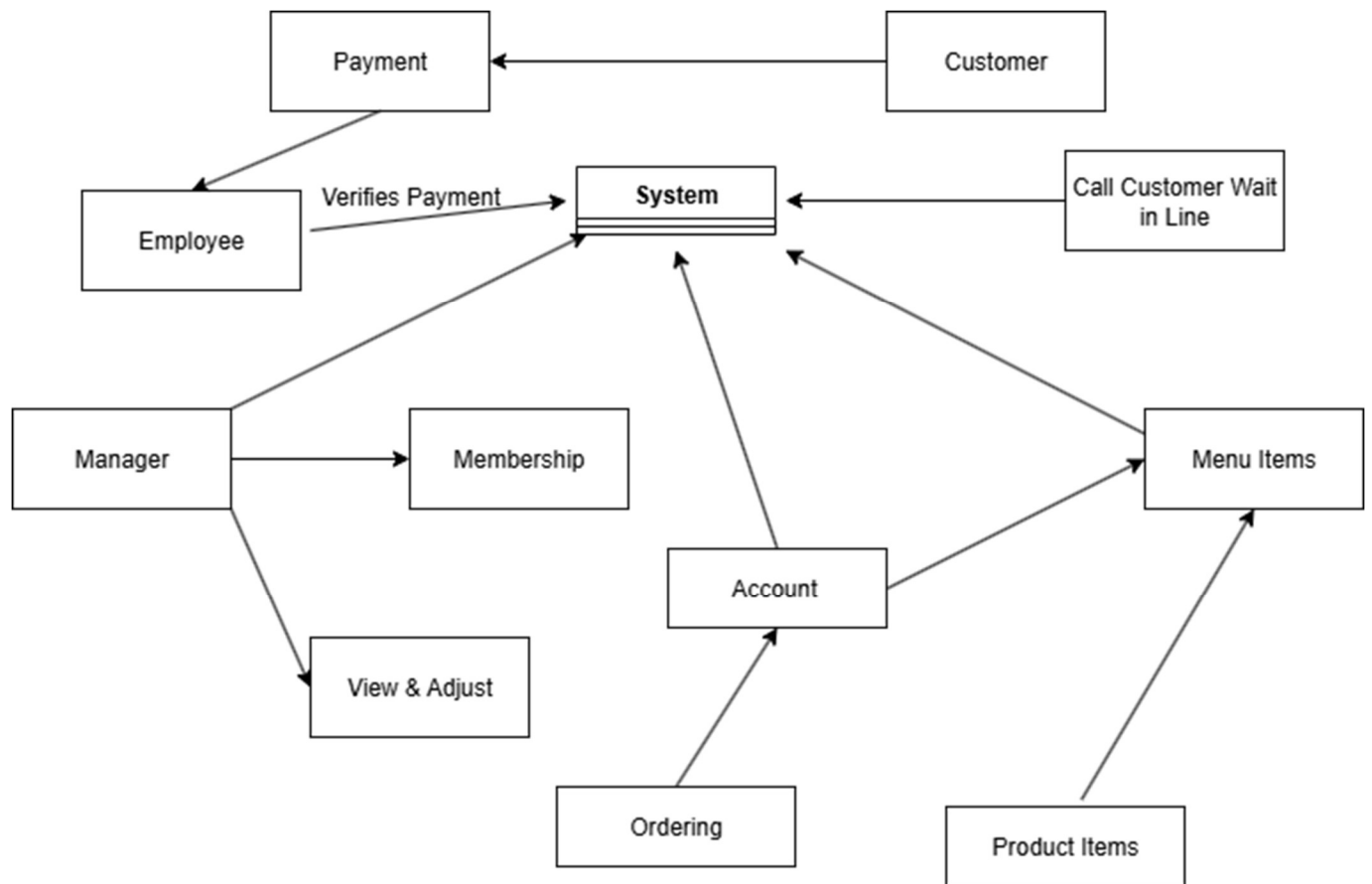
Manager Class



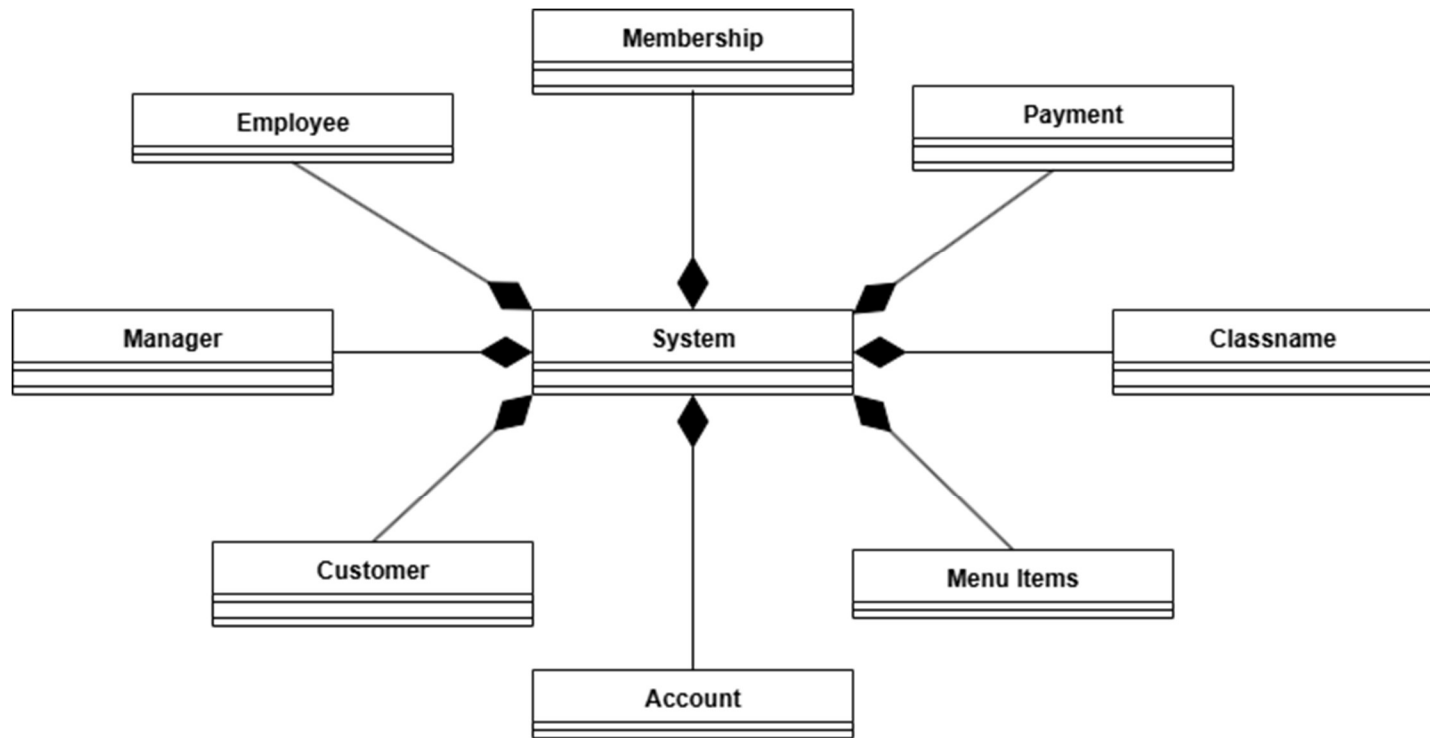
Employee Class



Graph of entities, attributes and showing relationship



System Composition



Brief Sketch of Class Diagram

Notes: I really enjoy the process of drawing out class diagrams, and the relationship between attributes and custom data types. Very nice and beautiful (:

